Frontend guide: roles and permissions

This guide covers the Access Control API implemented by the Go backend. It is for the frontend developer building the role editor and user-role assignment screens.

Important identifiers

- `role_id` is the stable lowercase role key, such as `staff`, `admin`, or

`auditor`. It is **not a UUID**. Use it in role URLs and store it as a string.

- `user_id` is the user's internal account UUID. Get it from the authenticated

session or the staff record's `account_id`. Do not use the official staff ID in the `/users/{userId}/roles` routes.

- Permission keys are stable strings such as `staff.read`, `roles.manage`, and

`audit.read`. Use backend-provided keys; do not generate keys from labels.

API base and session handling

Call the same-origin Next.js API proxy under `/api/v1`. The browser must send the server-managed session cookie; do not read, save, or send session tokens from JavaScript.

```
const response = await fetch("/api/v1/roles", {
  method: "GET",
  credentials: "include",
  cache: "no-store",
  headers: { Accept: "application/json" },
});
```

For JSON mutations, set `Content-Type: application/json`. The browser sends an `Origin` header on these state-changing requests; the backend rejects missing or cross-origin requests. Keep calls on the app's same origin and do not route around the Next.js API proxy.

Endpoints

Method and path	Permission	Purpose
`GET /api/v1/roles`	`roles.read`	List roles and their current permissions

Method and path	Permission	Purpose
`POST /api/v1/roles`	`roles.manage`	Create a role with no permissions
`PATCH /api/v1/roles/{role_id}`	`roles.manage`	Change a role's display name or description
`GET /api/v1/permissions`	`roles.read`	List the permission catalogue
`PUT /api/v1/roles/{role_id}/permissions`	`roles.manage`	Replace a role's complete permission set
`GET /api/v1/users/{user_id}/roles`	`roles.read`	Inspect an account's assigned role keys
`PUT /api/v1/users/{user_id}/roles`	`user_roles.assign`	Replace an account's complete role set

List roles

`GET /api/v1/roles`

```
{
  "roles": [
    {
      "role_id": "staff",
      "name": "Staff",
      "description": "Default least-privilege staff role",
      "is_system_role": true,
      "permissions": ["staff.read"]
    }
  ]
}
```

Use `is_system_role` to render system roles as read-only. The backend remains the authority and will reject edits even if the UI accidentally enables them.

Create or edit role details

`POST /api/v1/roles`:

```
{
  "role_id": "auditor",
  "name": "Auditor",
  "description": "Can review audit activity"
}
```

The response is `201 Created` with `{ "role": Role }`. The `role_id` must be a lowercase key containing letters, digits, or underscores and starting with a letter. New roles start with an empty permission list.

`PATCH /api/v1/roles/auditor` accepts one or both fields:

```
{
  "name": "Internal auditor",
  "description": "Reviews administrative activity"
}
```

The response is `200 OK` with the updated role and its current permissions. Role keys cannot be renamed. System roles cannot be edited.

Read and replace role permissions

`GET /api/v1/permissions` returns:

```
{
  "permissions": [
    { "permission_key": "audit.read", "description": "Search and view audit events" }
  ]
}
```

`PUT /api/v1/roles/auditor/permissions` replaces the full set, rather than adding to it:

```
{
  "permission_keys": ["audit.read", "staff.read"]
}
```

The response is `200 OK` with the saved `role_id` and `permission_keys`. Include every permission that should remain selected. Sending an empty array clears all permissions; do not send `null`.

The backend permits a caller to delegate only permissions they currently hold. It rejects unknown permissions and changes to system roles. A `permission_denied` response means the caller cannot delegate at least one of the requested permissions.

Read and replace user roles

`GET /api/v1/users/{user_id}/roles` returns:

```
{
  "user_id": "00000000-0000-4000-8000-000000000001",
  "role_ids": ["staff"]
}
```

`PUT /api/v1/users/{user_id}/roles` replaces all assigned roles:

```
{
  "role_ids": ["staff", "auditor"]
}
```

The response is `200 OK` with the complete saved role list. The frontend should reload the user's roles after a successful update rather than assuming the submitted values were applied.

The backend enforces that:

- The caller has `user_roles.assign`.
- The caller cannot assign roles to their own account.
- Every permission in every requested role is already held by the caller.
- The last active, unsuspended SuperAdmin cannot be removed.
- The role and target account exist.

An empty `role_ids` array removes all assignments if doing so does not remove the last active SuperAdmin. Do not send `null`.

Error handling

Errors use a consistent envelope:

```
{
  "error": {
    "code": "permission_denied",
    "request_id": "..."
  }
}
```

Handle at least these codes:

HTTP status	Example codes	Suggested UI behavior
`400`	`invalid_request`, `invalid_role`, `invalid_permission_set`, `invalid_user_id`	Show validation feedback; do not retry unchanged input
`401`	`unauthorized`	Clear frontend session state and return to sign-in
`403`	`permission_denied`, `invalid_origin`	Show an access/authorization message; do not retry
`404`	`not_found`	Explain that the role or account may have changed; refresh the view
`409`	`conflict`, `system_role_immutable`, `last_active_superadmin`	Explain the conflict and preserve the current form state

HTTP status	Example codes	Suggested UI behavior
`503`	`identity_unavailable`, `service_unavailable`	Show a retryable service error
`500`	`internal_error`	Show a generic failure and include the request ID in support details

For replacement requests, do not show success or update the table until the request succeeds. On an error, keep the user's unsaved selection available, show the error, and offer a deliberate retry or refresh.

UI authorization

Use the authenticated session's `permissions` to decide whether to show or enable controls:

- Show the role editor and permission catalogue to users with `roles.read`.
- Show role create/edit controls to users with `roles.manage`.
- Show role assignment controls to users with `user_roles.assign`.

These are presentation hints only. The Go API re-checks permissions on every request and applies delegation rules independently. Never treat a hidden button or a frontend role name as authorization.

Suggested screen loading sequence

1. Load the authenticated session.
2. If the session includes `roles.read`, request roles and permissions in

parallel.

1. If the session includes `user_roles.assign` and an account is selected,

load that account's assigned roles.

1. On mutation success, reload the affected role list or user-role set.
2. On `401`, use the existing authentication/session handling. On `403`,

retain current data and explain that the backend denied the operation.

API contract

The backend's OpenAPI definition is in `backend/contracts/openapi/access.yaml`.